Virtualization and containers are not mutually exclusive. You can use them both in a cloud native context, having an infrastructure made up of virtual machines on which you run containers. The Infrastructure as a Service (IaaS) model provides a virtualization layer that you can use to bootstrap new virtual machines. On top of that, you can install a container runtime and run your containers.

An application is usually made up of different containers that can be run on the same machine during development or while performing early testing. But you would quickly reach the point where it gets too complicated to manage many containers, primarily when you start replicating them for scalability and distributing them across different machines. That's when you'll start relying on the higher level of abstraction provided by the Container as a Service (CaaS) model, which offers functionality to deploy and manage containers in clusters of machines. Note that, behind the scenes, there is still a virtualization layer.

Containers are involved even when you're using PaaS platforms like Heroku or Cloud Foundry. You deploy your applications on those platforms by providing just the JAR artifact, since they take care of the JRE, the middleware, the OS, and any needed dependencies. Behind the scenes they build a container out of all those components and finally run it. The difference is that it's no longer you who is responsible for building a container—the platform does it for you. On the one hand, that's convenient, in terms of fewer responsibilities for developers. On the other hand, you are giving up control over the runtime and middleware, and you might face vendor lock-in.

In this book, you'll learn how to use Cloud Native Buildpacks (a CNCF project) to containerize Spring applications, and you'll use Docker to run them in your local environment.